

코흐 눈송이와 재귀 함수 탐구 활동지

[활동 1] 선 바꾸기 규칙 발견 & 직접 그려보기

프랙탈 도형은 단순한 규칙을 무한히 반복하여 만듭니다. 곧은 선분 1개를 3등분한 뒤 가운데를 뾰족하게 세운 4개의 선분으로 바꾸는 규칙을 손으로 직접 그려봅시다.

0단계 (기본 선분 1개)

1단계 (산 모양 4개 선분)

2단계 (직접 그려보기: 16개 선분)

1단계의 4개 선분 위에 각각 작은 산 그리기

[활동 2] 파이썬 터틀 명령어로 1단계 산 모양 만들기

길이 300인 선분 1개를 1단계 산 모양으로 그리기 위해 거북이를 어떻게 움직여야 할까요? 각 선분의 길이는 100입니다. 회전 각도와 이동 거리를 채워보세요.

1단계 산 모양 그리기 (총 4개 선분)
forward(100) # 1구간 직진
left() # 오르막 방향으로 좌회전
forward() # 2구간 직진 (오르막)
right() # 내리막 방향으로 우회전
forward(100) # 3구간 직진 (내리막)
left(60) # 수평 방향으로 좌회전
forward() # 4구간 직진 (마무리)

정삼각형 외각 회전 원리

[활동 3] 단계별 규칙성 표 채우기 & 한계 생각하기

초기 선분의 길이가 300일 때, 단계(depth)가 진행됨에 따라 변화하는 수치를 계산하여 표를 완성해 봅시다.

단계 (depth)	선분 1개의 개수	선분 1개의 길이	선분 전체 길이의 합
0단계	1개	300	300
1단계	4개 (1 × 4)	100 (300 ÷ 3)	400 (4 × 100)
2단계	개	약 33.33	
3단계	64개 (4³)		약 711.11

[생각 질문] 3단계나 4단계를 그리기 위해 [활동 2]의 코드를 복사해서 붙여넣는다면 어떤 문제가 발생할까요?

작성란:

코흐 곡선의 재귀 동작 원리와 눈송이 완성

[활동 4] 직진 대신 함수 부르기 - 재귀 함수로 압축하기

직진(forward)할 자리에 자기 자신 함수를 다시 호출하면 코드가 무한히 길어지지 않습니다. 빈칸을 채워 재귀 함수를 완성해 봅시다.

```
def koch(length, depth):
    # 1. 더 이상 쪼개지 않는 마지막 단계 (종료 조건)
    if depth == ____:
        forward(length)
    else:
        # 2. 길이를 3등분으로 축소
        length = length / ____
        # 3. forward 자리에 3분의 1 크기의 작은 산 함수 4회 호출
        koch(length, depth - 1)
        left(60)
        koch(length, ____ )
        right(120)
        koch(length, depth - 1)
        left(60)
        koch(length, depth - 1)
```

질문 1. `depth == 0` 조건이 없다면 어떻게 될까요?

작성란:

질문 2. `koch` 함수를 부를 때 `depth - 1`을 넘기는 이유는?

작성란:

[활동 5] 재귀 호출의 동작 과정 이해하기: `koch(300, 1)`의 실행 흐름

재귀 호출은 함수가 멈추고 부른 함수가 끝날 때까지 기다리는 과정입니다. `koch(300, 1)`이 실행될 때 일어나는 순서를 살펴봅시다.

`koch(300, 1)` 호출 시 컴퓨터의 실제 실행 순서:

- ① `koch(100, 0)` 호출 → `depth == 0`이므로 진짜로 **`forward(100)`** 직진 후 종료
- ② `left(60)` 회전
- ③ `koch(100, 0)` 호출 → `depth == 0`이므로 진짜로 **`forward(100)`** 직진 후 종료
- ④ `right(120)` 회전
- ⑤ `koch(100, 0)` 호출 → `depth == 0`이므로 진짜로 **`forward(100)`** 직진 후 종료
- ⑥ `left(60)` 회전
- ⑦ `koch(100, 0)` 호출 → `depth == 0`이므로 진짜로 **`forward(100)`** 직진 후 종료

치환(부품 교체)의 원리: `koch(300, 2)`를 실행하면 `koch(100, 1)`이 4번 호출됩니다. 즉, 단순한 선분 하나가 들어갈 자리에 길이 100짜리 산 모양 하나(_/__)가 통째로 갈아 끼워지면서 총 16개의 선분으로 나뉘게 됩니다.

[이해 확인] `koch(300, 1)`에서 화면에 실제로 선을 긋는 역할을 하는 유일한 단계(`depth` 값)는 무엇인가요?

작성란: `depth =`

[활동 6] 정삼각형의 3개 변을 모아 코흐 눈송이 완성하기

한 번에 완성한 코흐 곡선을 정삼각형의 3개 변에 모두 적용하여 달린 눈송이를 완성해 봅시다.

```
from turtle import *
shape('turtle')
speed(0)          # 최고 속도로 그리기

# 정삼각형 3개 변에 코흐 곡선 적용
for i in range(____):
    koch(300, 3)
    right(____)    # 정삼각형 외각 회전
```

